

# LEPL1503/LSINC1503 - Cours 2

*O. Bonaventure, B. Legat, M. Baerts*

☐ Full Width Mode    ☐ Present Mode

## ☰ **Table of Contents**

**Variables, pointeurs et doubles pointeurs**

**Matrices sur le heap**

**Initialisation d'une valeur**

**Libération de la mémoire**

**Un vecteur "Java" en C**

**Pointeur de fonction**

**Retourner un tableau**

**Structure différente du stack et heap**

# Variables, pointeurs et doubles pointeurs

## Visualisation

C (C17 + GNU extensions)

```
1 #include <stdlib.h>
2
➔ 3 int main(int argc, char **argv) {
4     int x;
5     int *px;
6     int **ppx;
7     x=1;
8     px=(int *) malloc(sizeof(int));
9     *px=2;
10    ppx=(int **) malloc(sizeof(int*)
11    *ppx=(int *) malloc(sizeof(int)
12    **ppx=3;
13 }
```

main

|      |                       |
|------|-----------------------|
| argc | int<br>?              |
| argv | pointer to char*<br>? |
| x    | int<br>?              |
| px   | pointer to int<br>?   |
| ppx  | pointer to int*<br>?  |

Stack

Heap

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

START: click Next > to run each line and see step-by-step visualization

< Prev

Next >

Step 1 of 7

Visualized with [pythontutor.com](https://pythontutor.com)

## Arguments de main

- int argc : Le nombre d'arguments (incluant le nom de l'exécutable)

- `char **argv` : liste des arguments initialisée par le système d'exploitation au lancement du programme

```
#include <stdio.h>
int main(int argc, char **argv)
{
    for (int i = 0; i < argc; i++)
    {
        printf("arg[%d]: %s\n", i, argv[i]);
    }
}
```

Running : `/tmp/jl\_p0QNq3/bin ab cd ef`

```
arg[0]: /tmp/jl_p0QNq3/bin
arg[1]: ab
arg[2]: cd
arg[3]: ef
```

## Matrices en C ⇄

---

Comment construire une matrice ?

```
float A[2][2] = {{1, 0}, {2, 3}};
A[1][1] = 4;
```

Une telle matrice sera stockée sur le **stack** (variable locale) ou dans la zone données (variable globale)

# Matrices sur le heap ⇌

---

## Comment mettre une matrice 2x2 sur le heap ? ⇌

=====

```
float *matrice1 = (float *)malloc(4 * sizeof(float));  
matrice1[1][1] = 4;
```



```
/tmp/jl_wjlCn8/main.c:5:14: error: subscripted value is not an array, pointer, or vector  
    5 |     matrice1[1][1] = 4;  
      |           ~~~~~^~  
1 error generated.
```



# Première approche, un malloc par ligne ⇌

C (C17 + GNU extensions)

```
1 #include <stdlib.h>
2
➔ 3 int main(int argc, char **argv) {
4     float **matrice2 = (float **)ma
5     float *ligne0 = (float *)malloc
6     float *ligne1 = (float *)malloc
7     matrice2[0] = ligne0;
8     matrice2[1] = ligne1;
9     matrice2[1][1] = 4;
10 }
```

line t

next

START: click **Next >** to run each line and see step-by-step visualization

< Prev

Next >

Step 1 of 7

Stack

Heap

main

argc

int

?

argv

pointer to char\*

?

matrice2

pointer to float\*

?

ligne0

pointer to float

?

ligne1

pointer to float

?

Note: ? refers to an uninitialized value

C/C++ details: none [default view] ▼

Visualized with [pythontutor.com](https://pythontutor.com)

# Deuxième approche, un malloc pour toutes les valeurs ⇌

C (C17 + GNU extensions)

```
1 #include <stdlib.h>
2
➔ 3 int main(int argc, char **argv) {
4     float **matrice3 = (float **)mal
5     float *valeurs = (float *)malloc
6     matrice3[0] = valeurs;
7     matrice3[1] = valeurs + 2;
8     matrice3[1][1] = 4;
9 }
```

main

|          |                        |
|----------|------------------------|
| argc     | int<br>?               |
| argv     | pointer to char*<br>?  |
| matrice3 | pointer to float*<br>? |
| valeurs  | pointer to float<br>?  |

Stack

Heap

START: click **Next >** to run each line and see step-by-step visualization

Note: ? refers to an uninitialized value

C/C++ [details](#): none [default view]

< Prev

Next >

Step 1 of 6

Visualized with [pythontutor.com](https://pythontutor.com)

# Initialisation d'une valeur ⇄

## Comment initialiser la première valeur ? ⇄

```
float **matrice = (float **)malloc(2 * sizeof(float *));
float *ligne0 = (float *)malloc(2 * sizeof(float));
float *ligne1 = (float *)malloc(2 * sizeof(float));
matrice[0] = ligne0;
matrice[1] = ligne1;
matrice[0][0] = 1; // 1
*(matrice[0]) = 1; // 2
*(matrice) = 1;   // 3
```



wooclap

<https://app.wooclap.com/JAPRXX>



## Comment faire `matrice[0][1] = 0` ? ⇄

```
float **matrice = (float **)malloc(2 * sizeof(float *));
float *ligne0 = (float *)malloc(2 * sizeof(float));
float *ligne1 = (float *)malloc(2 * sizeof(float));
matrice[0] = ligne0;
matrice[1] = ligne1;
*(matrice + 1) = 0; // 1
*(matrice++) = 0;   // 2
*(matrice[0] + 1) = 0; // 3
```



wooclap

<https://app.wooclap.com/JAPRXX>



## Comment faire `matrice[1][0] = 2` ? ⇐

=====

```
float **matrice = (float **)malloc(2 * sizeof(float *));  
float *ligne0 = (float *)malloc(2 * sizeof(float));  
float *ligne1 = (float *)malloc(2 * sizeof(float));  
matrice[0] = ligne0;  
matrice[1] = ligne1;  
*(*(++matrice)) = 2; // 1  
*(*(matrice + 1)) = 2; // 2  
*(*matrice + 1) = 2; // 3
```



**wooclap**

<https://app.wooclap.com/JAPRXX>





# Libération de la mémoire ↩

---

## L'importance de l'ordre de désallocation ↩

=====

Si la matrice a été créée par

```
float **matrice = (float **)malloc(2 * sizeof(float *));  
float *ligne0 = (float *)malloc(2 * sizeof(float));  
float *ligne1 = (float *)malloc(2 * sizeof(float));  
matrice[0] = ligne0;  
matrice[1] = ligne1;
```



► Comment libérer la mémoire avec free ?

**wooclap**

<https://app.wooclap.com/JAPRXX>



# Toujours free dans le sens inverse de malloc

C (C17 + GNU extensions)

```
1 #include <stdlib.h>
2
3 int main(int argc, char **argv) {
4     float **matrice = (float **)mal
5     matrice[0] = (float *)malloc(2
6     matrice[1] = (float *)malloc(2
7     free(matrice);
8     free(matrice[0]);
9     free(matrice[1]);
10 }
```

Stack

main

argc

int  
1

argv

pointer to char\*  


matrice

pointer to float\*  


Heap

ERROR on this step: click < Prev to see what led to it

< Prev

Next >

Done running (6 steps)

Note:  means a pointer points to memory that is either misaligned with data boundaries.  locations are *approx*i not match the pointer's real address. Select 'byte-level view' to see more details:

C/C++ details: none [default view] 

ERROR: Invalid read of size 8  
(Stopped running after the first error. Please fix your code.)

see [UNSUPPORTED FEATURES](#)undefined

# Un vecteur "Java" en C

---

## Première implémentation

---

```
struct vector_t
{
    int size;
    float *v;
};
```



```
struct vector_t * init(int size, float val) {
```



```
float get(struct vector_t *t, int i) {
```



```
void set(struct vector_t *t, int i, float val) {
```



```
void destroy(struct vector_t *vect) {
```

**wooclap**

<https://app.wooclap.com/JAPRXX>



## Une autre implémentation ...

---

Toutes les fonctions doivent vérifier leurs arguments et retourner

- -1 en cas d'erreur

- 0 en cas de succès

```
struct vector_t {  
    int length; // nombre d'éléments  
    float *tab; // tableau avec les réels  
};
```



## Initialisation ⇄

---

```
struct vector_t {  
    int length; // nombre d'éléments  
    float *tab; // tableau avec les réels  
};
```



Quels arguments ?

```
/*  
 * @pre  
 * alloue la mémoire pour un vecteur de size éléments  
 * initialisés à la valeur val  
 * @return -1 si erreur, 0 sinon  
 */  
  
int init(int size, float val,  
         struct vector_t v, // 1  
         struct vector_t **v, // 2  
         struct vector_t *v, // 3  
)
```



**wooclap**

<https://app.wooclap.com/JAPRXX>



## Allocation de la mémoire ⇄

---

```
struct vector_t {  
    int length; // nombre d'éléments  
    float *tab; // tableau avec les réels  
};
```



```
int init(int size, float val, struct vector_t **v)
{
    if ((size < 0) || v == NULL)
        return -1;
    *v = (struct vector_t *)malloc(sizeof(struct vector_t));
    if (*v == NULL)
    {
        return -1;
    }
    (*v)->tab = (float *)malloc(size * sizeof(float));
    if ((*v)->tab == NULL)
    {
        return -1;
    }
    (*v)->length = size;
    for (int i = 0; i < size; i++)
    {
```



► Comment assigner la valeur val au i ième élément ?

```
    }
    return 0;
}
```



**wooclap**

<https://app.wooclap.com/JAPRXX>



## Visualization de init ↗

C (C17 + GNU extensions)

```
15         return -1;
16     (*v)->tab = (float *)malloc(s
17     if ((*v)->tab == NULL)
18         return -1;
19     (*v)->length = size;
20     for (int i = 0; i < size; i++
21     {
22         float *t = (*v)->tab;
23         t[i] = val;
24         // ou *((*v)->tab + i) =
25     }
26     return 0;
27 }
28 int main()
➔ 29 {
30     struct vector_t *ptr = NULL;
31     int err = init(4, 1.23, &ptr)
32 }
```

Stack

main

ptr

pointer to vector\_t  
?

err

int  
?

Note: ? refers to an uninitialized value

C/C++ [details](#): 

none [default view] ▼

line t

next

START: click **Next >** to run each line and see  
step-by-step visualization

< Prev

Next >

Step 1 of 26

## Récupération d'un élément ↗

```
struct vector_t {
    int length; // nombre d'éléments
    float *tab; // tableau avec les réels
};
```



```
/*
 * @pre
 * @post retourne le ième élément du tableau dans val
 *         -1 en cas d'erreur, 0 sinon
 */

int get(struct vector_t *v, int i,
        float val // 1
        float **val // 2
        float *val // 3
) {
```



► Comment implémenter get ?

wooclap

<https://app.wooclap.com/JAPRXX>



## Modification d'un élément ↗

---

```
/*
 * @pre
 * @post place val comme ième élément
 *         du tableau
 *         -1 en cas d'erreur, 0 sinon
 */

int set(struct vector_t *v, int i,
        float val // 1
        float **val // 2
        float *val // 3
) {
```



► Comment implémenter set ?

wooclap

<https://app.wooclap.com/JAPRXX>



## Libération de la mémoire ↗

---

Comment libérer la mémoire quand le vecteur est devenu inutile ?

```
/*  
 * libère la mémoire utilisée pour le vecteur  
 * -1 en cas d'erreur, 0 sinon  
 */
```



```
int destroy(struct vector_t *v) {
```

**wooclap**

<https://app.wooclap.com/JAPRXX>





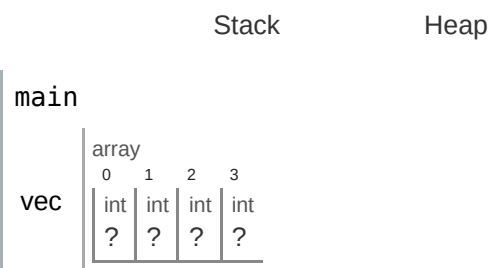
# Pointeur de fonction ↔

mapreduce donne un exemple de fonction d'ordre supérieur en C

C (C17 + GNU extensions)

```
9 {
10     return a + b;
11 }
12
13 int mapreduce(int (*f)(int), int
14 {
15     for (int i = 0; i < len; i++)
16     {
17         init = op(init, f(vec[i])
18     }
19     return init;
20 }
21
22 int main()
23 {
24     int vec[] = {1, 2, 3, 4};
25     printf("%d\\n", mapreduce(squ
26 }
```

Print output (drag lower right corner to resize)



**Note:** ? refers to an uninitialized value

C/C++ [details](#):

**START:** click **Next >** to run each line and see  
step-by-step visualization

< Prev

Next >

Step 1 of 43

Visualized with [pythontutor.com](https://pythontutor.com)

# Retourner un tableau ↩

---

## Heap ou stack ? ↩

.....

Laquelle de ces deux fonctions présente une façon correcte de retourner un tableau ?

```
int *stack()
{
    int v[] = {1, 2, 3};
    return v;
}

int *heap()
{
    int *v = (int *)malloc(3 * sizeof(int));
    v[0] = 1;
    v[1] = 2;
    v[2] = 3;
    return v;
}
```



**wooclap**

<https://app.wooclap.com/JAPRXX>



# Retourner un tableau : visualization

C (C17 + GNU extensions)

```
9 int *heap()  
10 {  
11     int *v = (int *)malloc(3 * si  
12     v[0] = 1;  
13     v[1] = 2;  
14     v[2] = 3;  
15     return v;  
16 }  
17  
18 int main()  
19 {  
20     int *v1 = heap();  
21     printf("%d %d %d\\n", v1[0],  
22     free(v1); // don't forget!  
23     int *v2 = stack();  
→ 24     printf("%d %d %d\\n", v2[0],  
25     return 0;  
26 }
```

Print output (drag lower right corner to resize)

1 2 3\\n

Stack

Heap

main

v1

pointer to int



v2

pointer to int

NULL (0x0)

**Note:**  means a pointer points to memory that is either misaligned with data boundaries.  locations are *approximate* not match the pointer's real address. Select 'byte-level view' to see more details:

C/C++ [details:](#)

**ERROR** on this step: click < **Prev** to see what led to it

< Prev

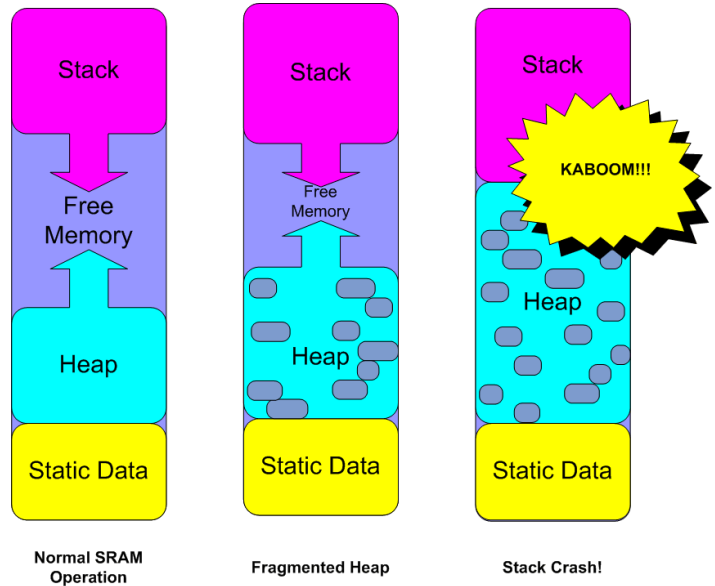
Next >

Done running (15 steps)

**ERROR: Invalid read of size 4**  
(Stopped running after the first error. Please fix your code.)

# Structure différente du stack et heap ↩

- Stack : Pile de mémoire locale des fonctions, structure connue à la **compilation**
- Heap : Alloué et désalloué avec des tailles connues à l'**exécution**. L'OS fait de son mieux pour organiser cette mémoire et réassigner les fragments formés suite aux désallocations



Source

The End

section (generic function with 1 method)

```
1 section(s) = md"# $s"
```

frametitle (generic function with 1 method)

```
1 frametitle(s) = md"## $s"
```

```
1 using PlutoUI, PlutoUI.ExperimentalLayout, PlutoTeachingTools, HypertextLiteral: @html
```

```
1 import HTTP, Clang_jll, MultilineStrings
```

compile\_lib (generic function with 2 methods)

```
1 begin
2 abstract type Code end
3
4 struct CCode <: Code
5     code::String
6 end
7
8 macro c_str(s)
9     return :($CCode($(esc(s))))
10 end
11
12 struct CppCode <: Code
13     code::String
14 end
15
16 macro cpp_str(s)
17     return :($CppCode($(esc(s))))
18 end
19
20 struct CLCode <: Code
21     code::String
22 end
23
24 macro cl_str(s)
25     return :($CLCode($(esc(s))))
26 end
27
28 struct JavaCode <: Code
29     code::String
30 end
31
32 macro java_str(s)
33     return :($JavaCode($(esc(s))))
34 end
35
36 source_extension(::CCode) = "c"
37 source_extension(::CppCode) = "cpp"
38 source_extension(::CLCode) = "cl"
39 source_extension(::JavaCode) = "java"
40
41 compiler(::CCode, mpi::Bool) = mpi ? "mpicc" : "clang"
42 function compiler(::CppCode, mpi::Bool)
43     @assert !mpi
44     return "clang++"
45 end
46
47 inline_code(code::AbstractString, ext::String) = HTML("""<code
class="language-$ext">$code</code>""")
```

```

48 inline_code(code::Code) = inline_code(code.code, source_extension(code))
49
50 function md_code(code::AbstractString, ext::String)
51     code = "`" * code * "`" * "\n" * code
52     if code[end] != '\n'
53         code *= '\n'
54     end
55     return Markdown.parse(code * "\n")
56 end
57 md_code(code::Code) = md_code(code.code, source_extension(code))
58 function Base.show(io::IO, m::MIME"text/html", code::Code)
59     return show(io, m, md_code(code))
60 end
61
62 function compile(
63     code::Code;
64     lib,
65     emit_llvm = false,
66     cflags = ["-O3"],
67     mpi::Bool = false,
68     use_system::Bool = mpi || "-fopenmp" in cflags, # '-fopenmp' will not work with
        pure Clang_jll, it needs openmp installed as well
69     verbose = 0,
70 )
71     path = mktempdir()
72     main_file = joinpath(path, "main." * source_extension(code))
73     bin_file = joinpath(path, ifelse(emit_llvm, "main.llvm", ifelse(lib, "lib.so",
        "bin")))
74     write(main_file, code.code)
75     args = String[]
76     if !use_system && code isa CppCode
77         # 'clang++' is not part of 'Clang_jll'
78         push!(args, "-x")
79         push!(args, "c++")
80     end
81     append!(args, cflags)
82     if lib
83         push!(args, "-fPIC")
84         push!(args, "-shared")
85     end
86     if emit_llvm
87         push!(args, "-S")
88         push!(args, "-emit-llvm")
89     end
90     push!(args, main_file)
91     push!(args, "-o")
92     push!(args, bin_file)
93     try
94         if use_system
95             cmd = Cmd([compiler(code, mpi); args])

```

```

96         if verbose >= 1
97             @info("Compiling : $cmd")
98         end
99         run(cmd)
100     else
101         Clang_jll.clang() do exe
102             cmd = Cmd([exe; args])
103             if verbose >= 1
104                 @info("Compiling : $cmd")
105             end
106             run(cmd)
107         end
108     end
109 catch err
110     if err isa ProcessFailedException
111         return
112     else
113         rethrow(err)
114     end
115 end
116 return bin_file
117 end
118
119 function emit_llvm(code; kws...)
120     llvm = compile(code; lib = false, emit_llvm = true, kws...)
121     InteractiveUtils.print_llvm(stdout, read(llvm, String))
122     return code
123 end
124
125 function compile_lib(code::Code; kws...)
126     return codesnippet(code), compile(code; lib = true, kws...)
127 end
128
129 function compile_and_run(code::Code; verbose = 0, args = String[], valgrind::Bool =
false, mpi::Bool = false, num_processes = nothing, show_run_command =
!isempty(args) || verbose >= 1, kws...)
130     bin_file = compile(code; lib = false, mpi, verbose, kws...)
131     if !isnothing(bin_file)
132         cmd_vec = [bin_file; args]
133         if valgrind
134             cmd_vec = ["valgrind"; cmd_vec]
135         end
136         if mpi
137             if !isnothing(num_processes)
138                 cmd_vec = [["-n", string(num_processes)]; cmd_vec]
139             end
140             cmd_vec = ["mpiexec"; cmd_vec]
141         end
142         cmd = Cmd(cmd_vec)
143         if show_run_command

```



```

144         @info("Running : $cmd") # '2:end-1' to remove the backsticks
145     end
146     try
147         run(cmd)
148     catch err
149         @warn(string(typeof(err)))
150     end
151 end
152 return codesnippet(code)
153 end
154
155 function wrap_in_main(content)
156     code = content.code
157     if code[end] == '\n'
158         code = code[1:end-1]
159     end
160     return typeof(content)("""
161 #include <stdlib.h>
162
163 int main(int argc, char **argv) {
164 $(MultilineStrings.indent(code, 2))
165 }
166 """)
167 end
168
169 function wrap_compile_and_run(code; kws...)
170     compile_and_run(wrap_in_main(code); kws...)
171     return code
172 end
173
174 # TODO It would be nice if the user could select a dropdown or hover with the mouse
175 # and see the full code
176 function codesnippet(code::Code)
177     lines = readlines(IOBuffer(code.code))
178     i = findfirst(line -> contains(line, "codesnippet"), lines)
179     if isnothing(i)
180         return code
181     end
182     j = findlast(line -> contains(line, "codesnippet"), lines)
183     return typeof(code)(join(lines[i+1:j-1], '\n'))
184 end
185
186 struct Example
187     name::String
188 end
189
190 function code(example::Example)
191     code = read(joinpath(dirname(dirname(dirname(@__DIR__))), "examples",
192 example.name), String)
193     ext = split(example.name, '.')[end]

```

```

192     if ext == "c"
193         return CCode(code)
194     elseif ext == "cpp" || ext == "cc"
195         return CppCode(code)
196     elseif ext == "cl"
197         return CLCode(code)
198     else
199         error("Unrecognized extension `$ext`.")
200     end
201 end
202
203 function compile_and_run(example::Example; kws...)
204     return compile_and_run(code(example); kws...)
205 end
206
207 function compile_lib(example::Example; kws...)
208     return compile_lib(code(example); kws...)
209 end

```

tutor (generic function with 1 method)

```

1  # No need to go beyond '620' because pythontutor will just add an internal slider
2  an not show full code
3  function tutor(code::Code; width = 800, height = min(620, 260 + 20 *
4      countlines(IOBuffer(code.code))), frameborder = 0, curInstr = 0)
5      esc_code = HTTP.escapeuri(code.code)
6      src = "https://pythontutor.com/iframe-
embed.html#code=$esc_code&cumulative=false&py=$(source_extension(code))&curInstr=$cu
rInstr"
7      return HTML("""<iframe width="$width" height="$height"
8          frameborder="$frameborder" src="$src"></iframe>""")
9  end

```

wooclap (generic function with 1 method)

```

1  function wooclap(link)
2      logo = HTML("""""")
4      url = "https://app.wooclap.com/$link"
5      a = HTML("""<a style="margin-left: 80px;" href="https://app.wooclap.com/$link">
6          <tt>$url</tt></a>""")
7      qr = img("https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=$url",
8          :height => 50, name = "$link.png")
9      return three_columns(logo, a, qr)
10 end

```

img (generic function with 3 methods)

```
1 begin
2   struct Path
3     path::String
4   end
5
6   function imgpath(path::Path)
7     file = path.path
8     if !('.' in file)
9       file = file * ".png"
10    end
11    return joinpath(joinpath(@__DIR__, "images", file))
12  end
13
14  function img(path::Path, args...; kws...)
15    return PlutoUI.LocalResource(imgpath(path), args...)
16  end
17
18  struct URL
19    url::String
20  end
21
22  function save_image(url::URL, html_attributes...; name = split(url.url, '/') [end],
23    kws...)
24    path = joinpath("cache", name)
25    return PlutoTeachingTools.RobustLocalResource(url.url, path,
26      html_attributes...), path
27  end
28
29  function img(url::URL, args...; kws...)
30    r, _ = save_image(url, args...; kws...)
31    return @html("<a href=$(url.url)>$r</a>")
32  end
33
34  function img(file::String, args...; kws...)
35    if startswith(file, "http")
36      img(URL(file), args...; kws...)
37    else
38      img(Path(file), args...; kws...)
39    end
40  end
41 end
```

three\_columns (generic function with 1 method)

```
1 three_columns(left, center, right) = hbox([
2     left,
3     Div(html" ", style = Dict("flex-grow" => "1")),
4     center,
5     Div(html" ", style = Dict("flex-grow" => "1")),
6     right,
7 ])
```

qa (generic function with 2 methods)

```
1 begin
2 function qa(question, answer)
3     return @html("<details><summary>$question</summary>$answer</details>")
4 end
5 function _inline_html(m::Markdown.Paragraph)
6     return sprint(Markdown.htmlinline, m.content)
7 end
8 function qa(question::Markdown.MD, answer)
9     # 'html(question)' will create '<p>' if 'question.content[]' is
'Markdown.Paragraph'
10    # This will print the question on a new line and we don't want that:
11    h = HTML(_inline_html(question.content[]))
12    return qa(h, answer)
13 end
14 end
```